

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 4

Study and Empirical Analysis of Algorithms:

Dijkstra and Floyd-Warshall (Dynamic Programming)

Elaborated:

st. gr. FAF-242

Bogdan Arama

Verified:

asist. univ.

Fiștic Cristofor

Summary

ALGORITHM ANALYSIS	3
Objective	3
Tasks	3
Theoretical Notes	3
Introduction	4
Comparison Metric	4
Input Format	4
SHORTEST PATH ALGORITHMS	5
Dijkstra's Algorithm	5
Floyd-Warshall Algorithm	6
EMPIRICAL RESULTS	7
Execution Time Comparison	7
Execution Time by Graph Density	8
Operations Count Comparison	9
CONCLUSION	10

ALGORITHM ANALYSIS

Objective

The objective of this laboratory work is to study the dynamic programming method of designing algorithms, implement Dijkstra's algorithm and the Floyd-Warshall algorithm, perform empirical analysis on sparse and dense weighted graphs, and present the results through graphical visualizations.

Tasks

1. Study the dynamic programming method of designing algorithms;
2. Implement Dijkstra and Floyd-Warshall in a programming language;
3. Perform empirical analysis of these algorithms for sparse and dense graphs;
4. Increase the number of nodes and analyze how this influences performance;
5. Present the obtained data graphically and deduce conclusions.

Theoretical Notes

Dynamic programming solves problems by breaking them into overlapping subproblems, storing partial results, and reusing them to build an optimal global solution. For shortest paths, optimal substructure allows computing shortest distances stage by stage.

Dijkstra's algorithm solves the **single-source shortest path** problem on graphs with non-negative edge weights. It greedily extends a set of vertices whose shortest distance from the source is already known. With a binary heap, its complexity is $O((V + E) \log V)$.

The Floyd-Warshall algorithm solves the **all-pairs shortest path** problem using dynamic programming. It iteratively allows paths through intermediate vertices $0, 1, \dots, V - 1$ and updates distances in a $V \times V$ matrix. Its time complexity is $\Theta(V^3)$ and does not depend on edge sparsity in the standard implementation.

Introduction

Shortest path problems appear in navigation systems, network routing, logistics, and game AI. Dijkstra is preferred when one source and non-negative weights are required. Floyd-Warshall is used when distances between all pairs of vertices are needed and the graph is small or dense enough that $O(V^3)$ is acceptable.

This laboratory compares both approaches on the same weighted graph instances, using sparse graphs ($E \approx V$) and dense graphs ($E \approx V \log_2 V$), while increasing the number of vertices.

Comparison Metric

The primary metric is execution time $T(V)$ in seconds (`perf_counter`), with three runs per measurement and the minimum kept.

A secondary metric is the number of **operations**: edge relaxations for Dijkstra and inner-loop updates for Floyd-Warshall.

Input Format

Graphs are **weighted** and **undirected**. Edge weights are random integers from 1 to 100. Representation:

- Dijkstra: adjacency list with pairs (`neighbor`, `weight`);
- Floyd-Warshall: adjacency matrix with `inf` when no edge exists.

Vertex sizes: 50, 100, 150, $\dots$, 500. Two families:

- **Sparse:** $E \approx V$;
- **Dense:** $E \approx V \cdot \log_2 V$.

Source vertex for Dijkstra: node 0.

SHORTEST PATH ALGORITHMS

1. Dijkstra's Algorithm

Overview

Dijkstra computes shortest distances from a source vertex to all others. It uses a priority queue (min-heap) to always process the unvisited vertex with the smallest tentative distance.

How It Works

Initialize distance to the source as 0 and all others as infinity. Repeatedly extract the minimum-distance vertex, relax all outgoing edges, and push improved distances into the heap. When edge weights are non-negative, each vertex is settled once.

Algorithm 1 Dijkstra(G , start)

```
1:  $dist[start] \leftarrow 0$ , all other  $dist \leftarrow \infty$ 
2: push  $(0, start)$  into min-heap
3: while heap not empty do
4:    $(d, u) \leftarrow$  pop minimum
5:   if  $d > dist[u]$  then continue
6:   end if
7:   for each edge  $(u, v, w)$  do
8:     if  $dist[u] + w < dist[v]$  then
9:        $dist[v] \leftarrow dist[u] + w$ 
10:      push  $(dist[v], v)$ 
11:    end if
12:  end for
13: end while
```

Listing 1: Dijkstra implementation (algorithms/dijkstra.py)

```
1 import heapq
2
3
4 def dijkstra(adjacency_list, start=0):
5     """
6     Single-source shortest paths (non-negative weights).
7     adjacency_list[u] = list of (neighbor, weight).
8     Returns (distances, operations_count).
9     """
```

```

10     n = len(adjacency_list)
11     if n == 0:
12         return [], 0
13
14     start = start % n
15     distances = [float("inf")] * n
16     distances[start] = 0
17     operations = 0
18
19     heap = [(0, start)]
20
21     while heap:
22         current_dist, node = heapq.heappop(heap)
23         if current_dist > distances[node]:
24             continue
25
26         for neighbor, weight in adjacency_list[node]:
27             operations += 1
28             new_dist = current_dist + weight
29             if new_dist < distances[neighbor]:
30                 distances[neighbor] = new_dist
31                 heapq.heappush(heap, (new_dist, neighbor))
32
33     return distances, operations

```

Complexity

- **Time:** $O((V + E) \log V)$ with a binary heap.
- **Space:** $O(V)$.

2. Floyd-Warshall Algorithm

Overview

Floyd-Warshall finds shortest paths between every pair of vertices. It is a classic dynamic programming algorithm where subproblem $dp^{(k)}[i][j]$ is the shortest path from i to j using only intermediate vertices from $\{0, \dots, k\}$.

How It Works

Start with direct edge weights. For each intermediate vertex k , update all pairs (i, j) : if passing through k is shorter, replace $dist[i][j]$.

Algorithm 2 Floyd-Warshall(*dist* matrix)

```
1: for  $k \leftarrow 0$  to  $V - 1$  do
2:   for  $i \leftarrow 0$  to  $V - 1$  do
3:     for  $j \leftarrow 0$  to  $V - 1$  do
4:        $dist[i][j] \leftarrow \min(dist[i][j], dist[i][k] + dist[k][j])$ 
5:     end for
6:   end for
7: end for
```

Listing 2: Floyd-Warshall implementation (algorithms/floyd_warshall.py)

```
1 def floyd_warshall(weight_matrix):
2     """
3     All-pairs shortest paths using dynamic programming.
4     weight_matrix[i][j] = edge weight, inf if no edge, 0 on
5     diagonal.
6     Returns (distance_matrix, operations_count).
7     """
8     n = len(weight_matrix)
9     if n == 0:
10         return [], 0
11
12     dist = [row[:] for row in weight_matrix]
13     operations = 0
14
15     for k in range(n):
16         for i in range(n):
17             row_ik = dist[i][k]
18             for j in range(n):
19                 operations += 1
20                 through_k = row_ik + dist[k][j]
21                 if through_k < dist[i][j]:
22                     dist[i][j] = through_k
23
24     return dist, operations
```

Complexity

- **Time:** $\Theta(V^3)$.
- **Space:** $\Theta(V^2)$ for the distance matrix.

EMPIRICAL RESULTS

Benchmarks were executed with `main.py`. Table 1 shows results at $V = 500$.

Table 1: Benchmark results at $V = 500$.

Type	V	E	Dijkstra (s)	Floyd (s)	Floyd ops
sparse	500	500	0.000282	13.075116	125000000
dense	500	4482	0.002161	19.238163	125000000

Execution Time Comparison

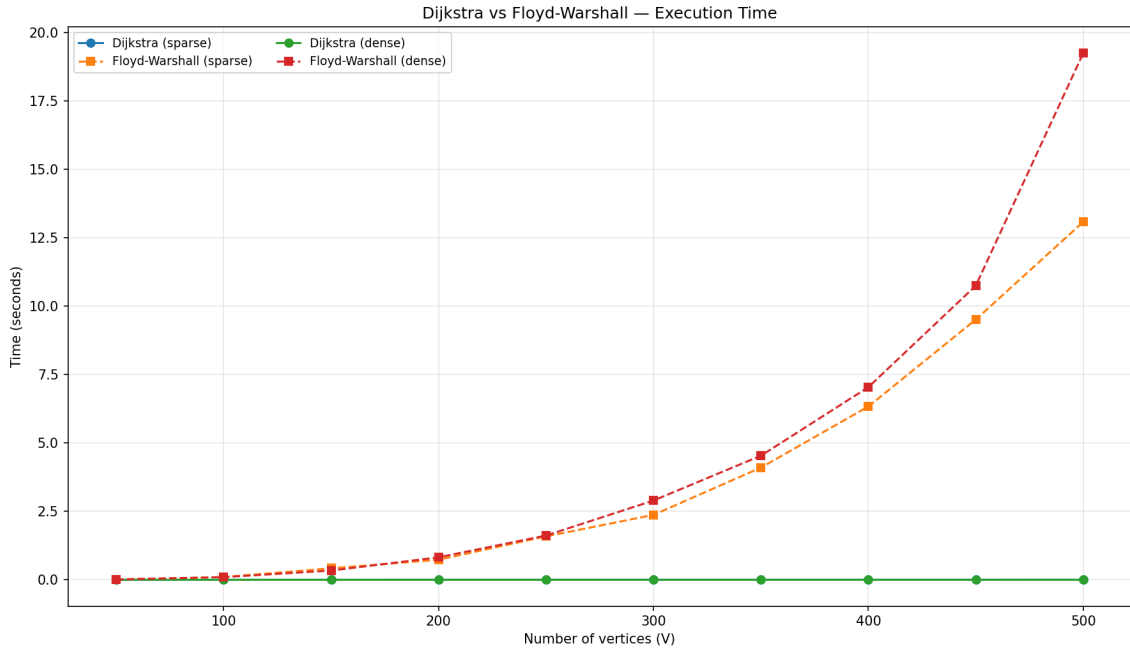


Figure 1: Dijkstra vs Floyd-Warshall — execution time (sparse and dense).

Figure 1 shows that Floyd-Warshall grows much faster with V because of its $O(V^3)$ behavior. Dijkstra remains very fast on both sparse and dense graphs within the tested range. Dense graphs increase Dijkstra's runtime due to more relaxations (E is larger), but the difference is small compared to Floyd-Warshall.

Execution Time by Graph Density

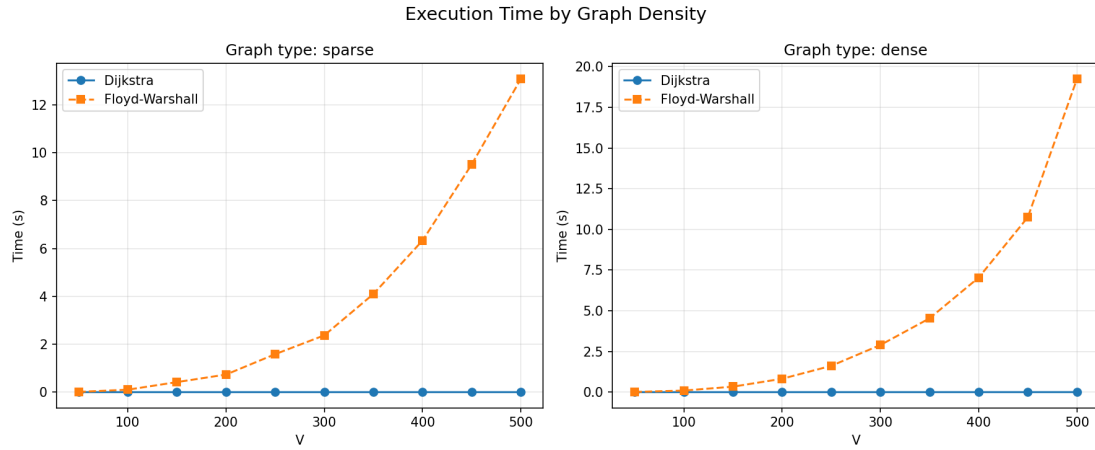


Figure 2: Execution time by graph type (sparse vs dense).

Figure 2 highlights that graph density affects Dijkstra noticeably (more edges to relax), while Floyd-Warshall timings are almost identical for sparse and dense graphs with the same V , since the algorithm always processes the full $V \times V$ matrix.

Operations Count Comparison

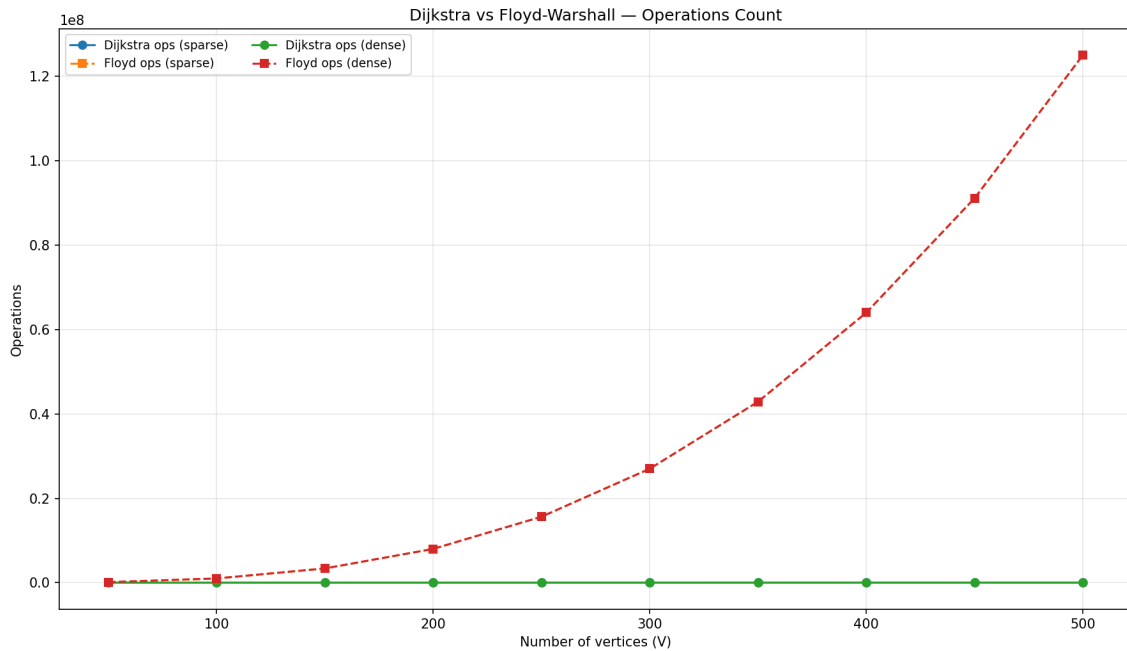


Figure 3: Dijkstra vs Floyd-Warshall — operations count.

Figure 3 confirms that Floyd-Warshall performs exactly V^3 updates per run (e.g. 125×10^6 at $V = 500$). Dijkstra's operation count grows with E and is much smaller on sparse graphs.

CONCLUSION

The empirical analysis compared Dijkstra’s algorithm and the Floyd-Warshall algorithm on weighted sparse and dense graphs with increasing vertex counts.

Dijkstra demonstrated excellent practical performance for single-source shortest paths. Its runtime grew slowly with V and increased moderately on dense graphs because of the larger number of edges. This matches the theoretical $O((V + E) \log V)$ behavior.

Floyd-Warshall showed the characteristic cubic growth of dynamic programming all-pairs shortest paths. At $V = 500$, Floyd-Warshall required about 13–19 seconds versus less than 0.003 seconds for Dijkstra, because the standard implementation always executes $\Theta(V^3)$ work regardless of sparsity.

Graph density strongly influences Dijkstra but has little effect on Floyd-Warshall in our implementation, since the matrix-based algorithm does not exploit sparsity. For large graphs where only one source is needed, Dijkstra is clearly preferable. Floyd-Warshall is appropriate for small graphs when a full distance matrix is required.

In summary, dynamic programming in Floyd-Warshall trades simplicity and completeness (all pairs) for high computational cost, while Dijkstra remains the efficient choice for single-source problems on sparse and dense networks with non-negative weights.

REFERENCES

1. GeeksforGeeks — *Dynamic Programming*.
[geeksforgeeks.org/dynamic-programming](https://www.geeksforgeeks.org/dynamic-programming)
2. TutorialsPoint — *Dynamic Programming*.
[tutorialspoint.com/dynamic-programming](https://www.tutorialspoint.com/dynamic-programming)
3. Coding Ninjas — *Dijkstra's Algorithm vs Floyd-Warshall's Algorithm*.
[codingninjas.com](https://www.codingninjas.com)

Access the full implementation and analysis on GitHub:

<https://github.com/bogdan456tech/aa-labs/tree/main/lab4>